Multinomial Model

General Principles

To model the relationship between a vector outcome variable in which each element of the vector is a frequency from a set of more than two categories and one or more independent variables, we can use a *Multinomial* model.

Considerations

> **Note**
>
> - We have the same considerations as for the Categorical model.

Example

Below is an example code snippet demonstrating a Bayesian multinomial model using the BayesForge (BF) package. This example is based on McElreath (2018).

Python

```python
from BayesForge import bf
import jax.numpy as jnp
import jax
import numpy as np

# Setup device -------------------------------------------------
m = bf('cpu')

# Simulated data -----------------------------------------------
np.random.seed(42)
```

```
N = 200
n_trials = 20
X = np.random.normal(0, 1, size=N)

true_alpha = np.array([0.5, -0.5])
true_beta = np.array([1.0, 0.5])

phi = np.column_stack([
    true_alpha[0] + true_beta[0] * X,
    true_alpha[1] + true_beta[1] * X,
    np.zeros(N),
])
theta = np.asarray(jax.nn.softmax(jnp.array(phi), axis=1))
Y = np.array([np.random.multinomial(n_trials, theta[i]) for i in range(N)])

m.data_on_model = dict(X=jnp.array(X), Y=jnp.array(Y))

# Define model ------------------------------------------------
def model(X, Y):
    alpha = m.dist.normal(0, 1, shape=(2,), name='alpha')
    beta = m.dist.normal(0, 1, shape=(2,), name='beta')
    s1 = alpha[0] + beta[0] * X
    s2 = alpha[1] + beta[1] * X
    s3 = jnp.zeros_like(s1)
    phi = jnp.stack([s1, s2, s3], axis=1)
    theta = jax.nn.softmax(phi, axis=1)
    m.dist.multinomial(total_count=20, probs=theta, obs=Y)

# Run sampler -------------------------------------------------
m.fit(model, progress_bar=False)

# Summary -----------------------------------------------------
m.summary()
```

```
An NVIDIA GPU may be present on this machine, but a CUDA-enabled jaxlib is not installed. Fal

bf v 0.0.60 package loaded
jax.local_device_count 16
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
alpha[0]	0.51	0.04	0.44	0.57	3046.60	2951.12	1.0	0.0	0.0
alpha[1]	-0.49	0.05	-0.57	-0.41	2935.93	2990.46	1.0	0.0	0.0
beta[0]	1.05	0.05	0.98	1.13	3087.95	2897.44	1.0	0.0	0.0
beta[1]	0.50	0.06	0.41	0.59	2818.30	2695.54	1.0	0.0	0.0

R

```r
library(BayesForge)
jax = reticulate::import('jax')
jnp <- reticulate::import('jax.numpy')
# Setup platform-------------------------------------------------
m=importBF(platform='cpu')

# Simulated data -----------------------------------------------
set.seed(42)
N = 200L
n_trials = 20L
X = rnorm(N, 0, 1)

true_alpha = c(0.5, -0.5)
true_beta = c(1.0, 0.5)

phi = cbind(true_alpha[1] + true_beta[1] * X,
            true_alpha[2] + true_beta[2] * X,
            rep(0.0, N))
exp_phi = exp(phi)
theta = exp_phi / rowSums(exp_phi)

Y = matrix(0L, nrow=N, ncol=3L)
for (i in 1:N) {
  Y[i, ] = as.integer(rmultinom(1, size = n_trials, prob = theta[i, ]))
}

m$data_on_model = list(X = jnp$array(as.numeric(X)), Y = jnp$array(Y))

# Define model -------------------------------------------------
model <- function(X, Y){
  alpha = m$dist$normal(0, 1, name='alpha', shape = reticulate::tuple(2L))
  beta = m$dist$normal(0, 1, name='beta', shape = reticulate::tuple(2L))
```

```
  s1 = alpha[0L] + beta[0L] * X
  s2 = alpha[1L] + beta[1L] * X
  s3 = jnp$zeros_like(s1)

  phi = jnp$stack(list(s1, s2, s3), axis = 1L)
  theta = jax$nn$softmax(phi, axis = 1L)

  m$dist$multinomial(total_count = 20L, probs = theta, obs = Y, name = "Y")
}

# Run sampler ----------------------------------------------
m$fit(model, progress_bar=FALSE)

# Summary --------------------------------------------------
m$summary()
```

Julia

```
using PyBayesForge
using Random
using Distributions

# Setup device---------------------------------------------------
m = importBF(platform="cpu")

# Simulated data -------------------------------------------------
Random.seed!(42)
N = 200
n_trials = 20
X = rand(Normal(0, 1), N)

true_alpha = [0.5, -0.5]
true_beta = [1.0, 0.5]

Y = zeros(Int, N, 3)
for i in 1:N
    phi1 = true_alpha[1] + true_beta[1] * X[i]
    phi2 = true_alpha[2] + true_beta[2] * X[i]
    e = exp.([phi1, phi2, 0.0])
      = e / sum(e)
```

```
    Y[i, :] = rand(Multinomial(n_trials, ))
end

m.data_on_model = pydict("X" => jnp.array(X), "Y" => jnp.array(Y))

# Define model ------------------------------------------------
@BF function model(X, Y)
    alpha = m.dist.normal(0, 1, shape=(2,), name="alpha")
    beta = m.dist.normal(0, 1, shape=(2,), name="beta")
    s1 = alpha[0] + beta[0] * X
    s2 = alpha[1] + beta[1] * X
    s3 = jnp.zeros_like(s1)
    phi = jnp.stack(pylist([s1, s2, s3]), axis=1)
    theta = jax.nn.softmax(phi, axis=1)
    m.dist.multinomial(total_count=20, probs=theta, obs=Y)
end

# Run mcmc ----------------------------------------------------
m.fit(model, progress_bar=false)

# Summary -----------------------------------------------------
m.summary()
```

Mathematical Details

We can model a vector of frequencies using a Dirichlet distribution. For an outcome variable Y_i with K categories, the *Dirichlet* likelihood function is:

$$Y_i \sim \text{Multinomial}(\theta_i)\theta_i = \text{Softmax}(\phi_i)\phi_{[i,1]} = \alpha_1 + \beta_1 X_i \phi_{[i,2]} = \alpha_2 + \beta_2 X_i ... \phi_{[i,k]} = 0\alpha_k \sim \text{Normal}(0, 1)\beta_k \sim \text{Norm}$$

Where:

- Y_i is the outcome (i.e. the vector of frequencies for each k categories) for observation i.
- θ_i is a vector unique to each observation, i, which gives the probability of observing i in category k.
- ϕ_i give the linear model for each of the k categories. Note that we use the softmax function to ensure that that the probabilities θ_i form a simplex .

- Each element of ϕ_i is obtained by applying a linear regression model with its own respective intercept α_k and slope coefficient β_k. To ensure the model is identifiable, one category, K, is arbitrarily chosen as a reference or baseline category. The linear predictor for this reference category is set to zero. The coefficients for the other categories then represent the change in the log-odds of being in that category versus the reference category.

Model validation tests

- Comparison with Stan: `8b.Multinomial fixed income.py` fits the same model with BayesForge and Stan and compares the posterior distributions (KL divergence) and parameter recovery.
- Documentation examples: the Python, R and Julia examples of this page are run by the documentation tests (`Python/test_model_pages.py`, `R/test_r_model_pages.py`, `Julia/test_julia_model_pages.py`).

Reference(s)

McElreath, Richard. 2018. *Statistical Rethinking: A Bayesian course with examples in R and Stan.* Chapman; Hall/CRC.